

Astro 142

Discussion 2

2025-10-02
Dyson Lewis

Short Answer Questions

Describe the process of code compilation

Preprocessing

- Handles directives starting with # (like #include, #define, #ifdef).
- Expands macros, includes header files, and removes comments.

Compilation

- Translates the preprocessed C code into assembly language for the target machine.
- Performs syntax checking and optimizations.

Assembly

- Converts the assembly code into machine code (object code).

Linking

- Combines your object file with libraries and other object files.

Compare static and dynamic linking

Static linking: library code is copied into the executable $\Rightarrow$ bigger file, no external dependencies, but must recompile to update libraries.

Dynamic linking: executable references shared libraries at runtime $\Rightarrow$ smaller file, updates without recompiling, but needs correct libraries installed.

Compare static and dynamic code interpretation

Static (ahead-of-time) interpretation/compilation

- Code is analyzed and translated before execution.
- Usually faster at runtime since work is done ahead.

Dynamic interpretation

- Code is analyzed and executed at runtime (line by line or block by block)
- More flexible — can run code generated on the fly.
- Slower, since parsing/execution happen together.

Discussion Activity

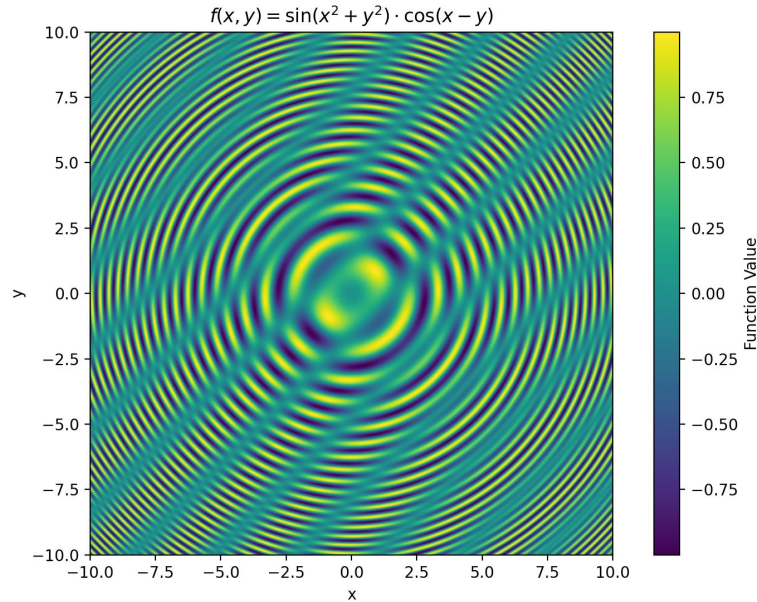
Discussion 2 Activity

1. Download the tar file from BruinLearn. Extract the demo files with proper tar commands.
2. Try manually compiling/running the c code and manually running the python code [demo1.py](#). Then try using the makefile to do the same.
3. Do make clean.
4. Try manually running the python code [demo2.py](#). Then, modify the makefile to run everything including [demo2.py](#).
5. Screenshot your terminal for each step and paste on the next slides. Make sure to include a screenshot of your final makefile.

Running demo1 and using Makefile

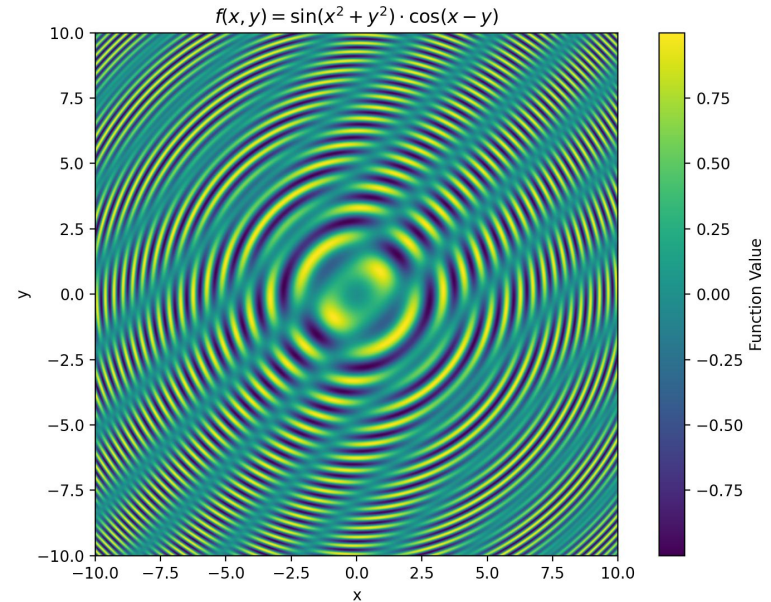
```
dyson@dpl-linux: ~/fall25/142ASTR/week1/disc2/demo
dyson@dpl-linux:~/fall25/142ASTR/week1/disc2/demo$ gcc helloworld.c -o helloworld
dyson@dpl-linux:~/fall25/142ASTR/week1/disc2/demo$ ./helloworld
Hello, World!
This is a custom function.
dyson@dpl-linux:~/fall25/142ASTR/week1/disc2/demo$ python3 demo1.py
```

Figure 1



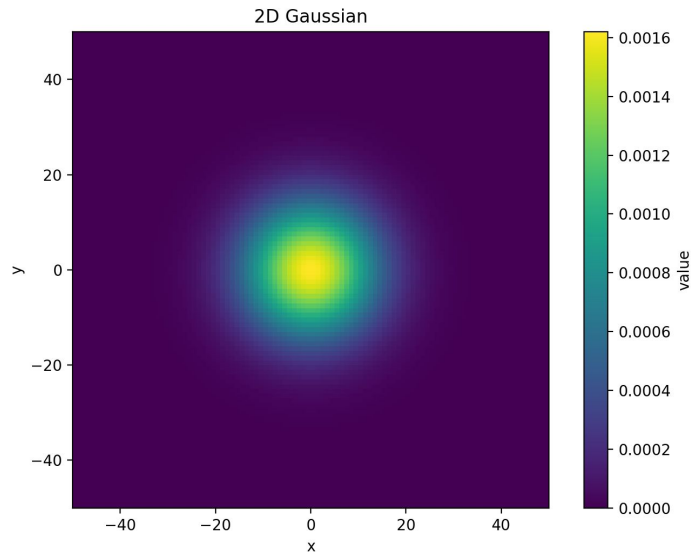
```
dyson@dpl-linux: ~/fall25/142ASTR/week1/disc2/demo
dyson@dpl-linux:~/fall25/142ASTR/week1/disc2/demo$ make
gcc helloworld.c -o helloworld
./helloworld
Hello, World!
This is a custom function.
python demo1.py
```

Figure 1



Running demo2, and modifying Makefile

```
dyson@dpl-linux: ~/fall25/142ASTR/week1/disc2/demo
dyson@dpl-linux:~/fall25/142ASTR/week1/disc2/demo$ python3 demo2.py
```



```
dyson@dpl-linux: ~/fall25/142
dyson@dpl-linux:~/fall25/142ASTR/week1/disc2/demo$ cat Makefile
all: hello pydemo

helloworld: helloworld.c
    gcc helloworld.c -o helloworld

hello: helloworld
    ./helloworld

pydemo: demo1.py demo2.py
    python demo1.py &
    python demo2.py

clean:
    rm helloworld
    rm -f *.pdf
    rm -f *.dat
```

Using makefile to run demo1 and demo2

